

Verification using UVM : Testbench Architecture

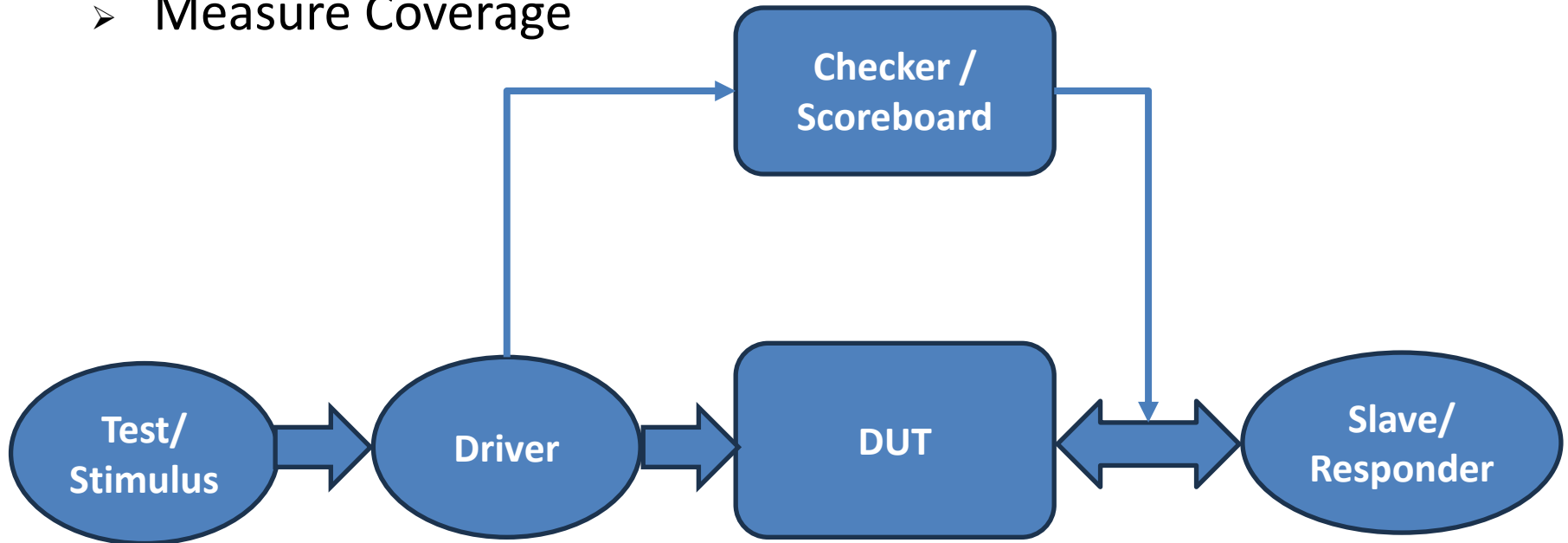
Pravin Zode

Outline

- Traditional Testbench architecture
- Test and Testbench

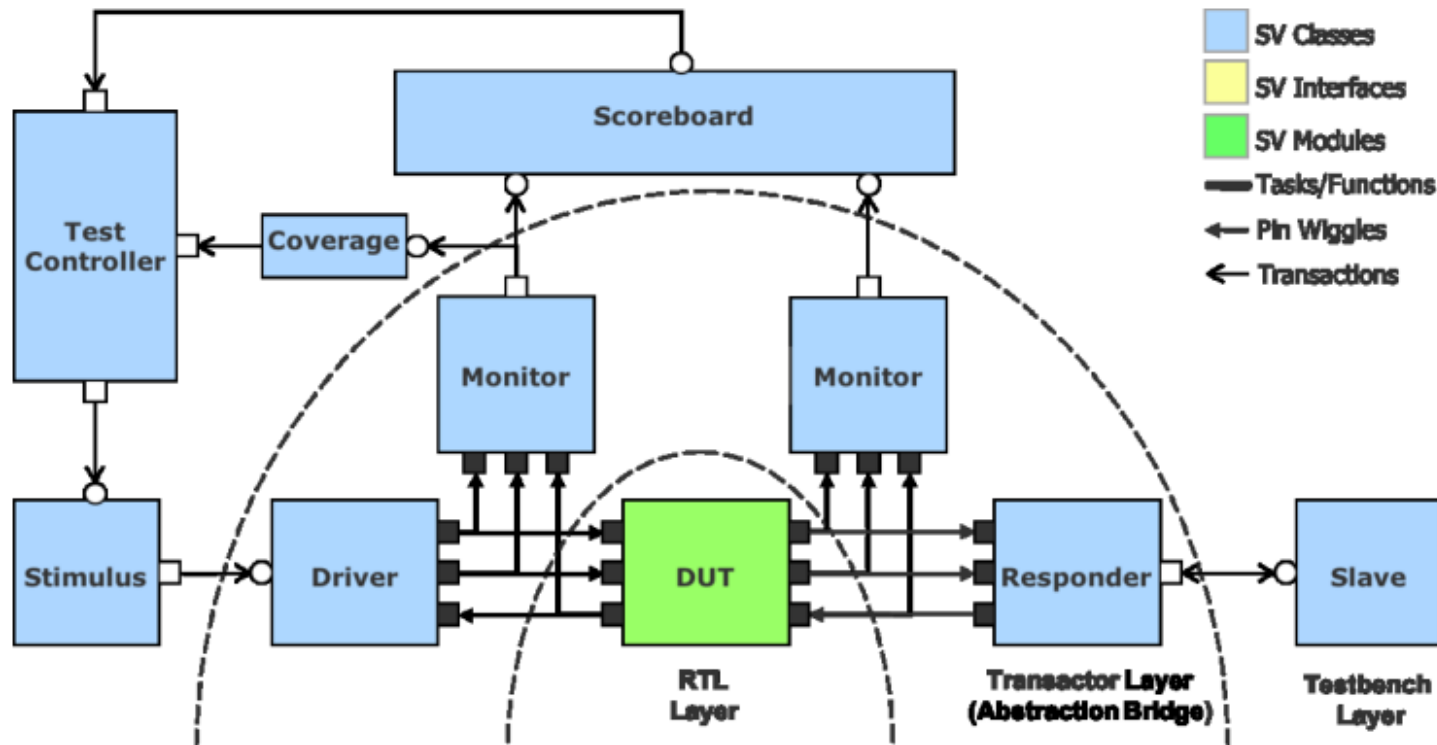
Traditional Testbench Architecture

- A testbench is a verification environment used to
 - Generate stimulus for DUT
 - Apply inputs to DUT
 - Observe outputs
 - Check Correctness
 - Measure Coverage

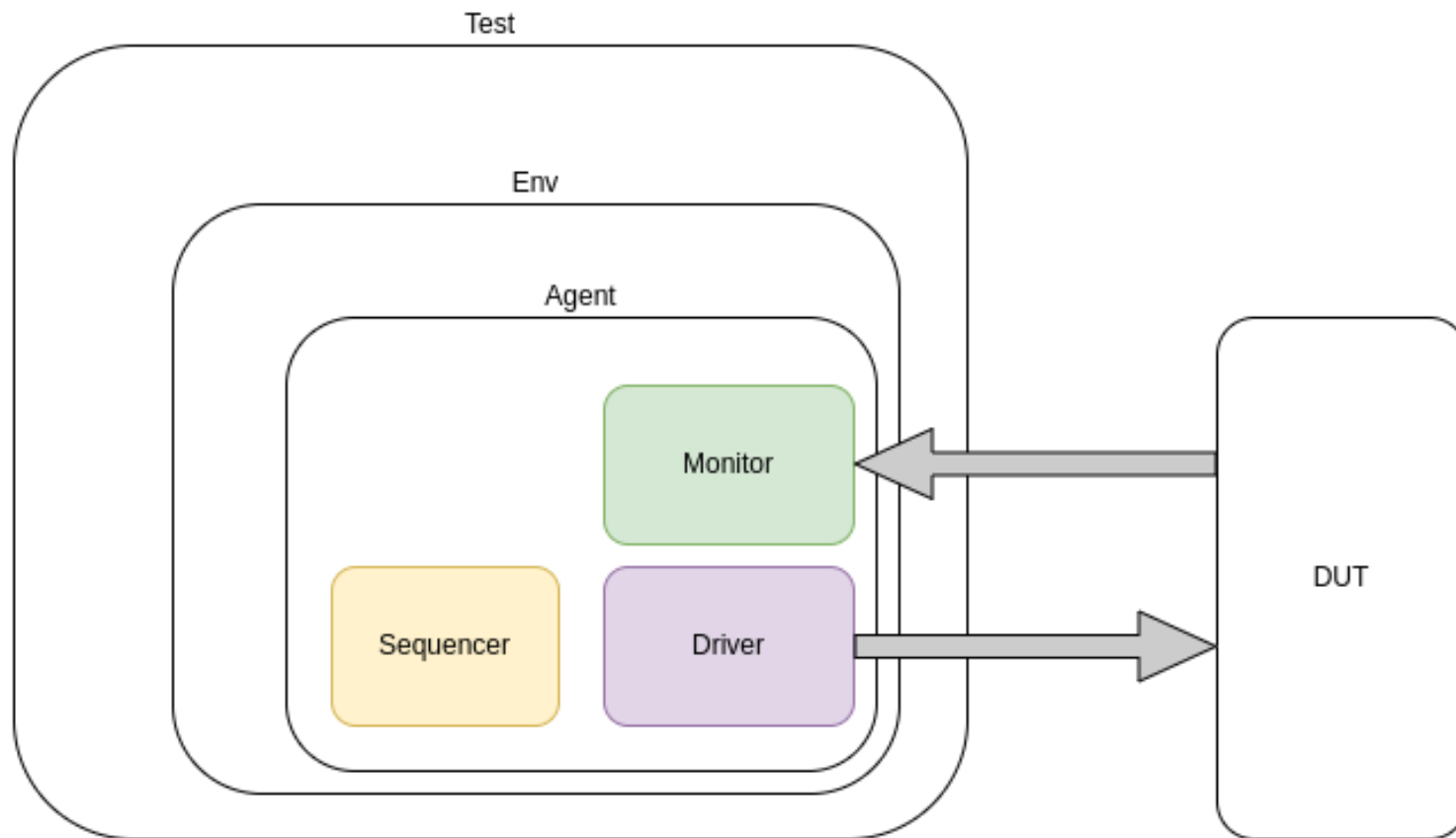


Layered Testbench Architecture

- Structured testbenches are important for complex DUT verification
 - Better maintainability
 - Better reusability
 - Parallel development and verification

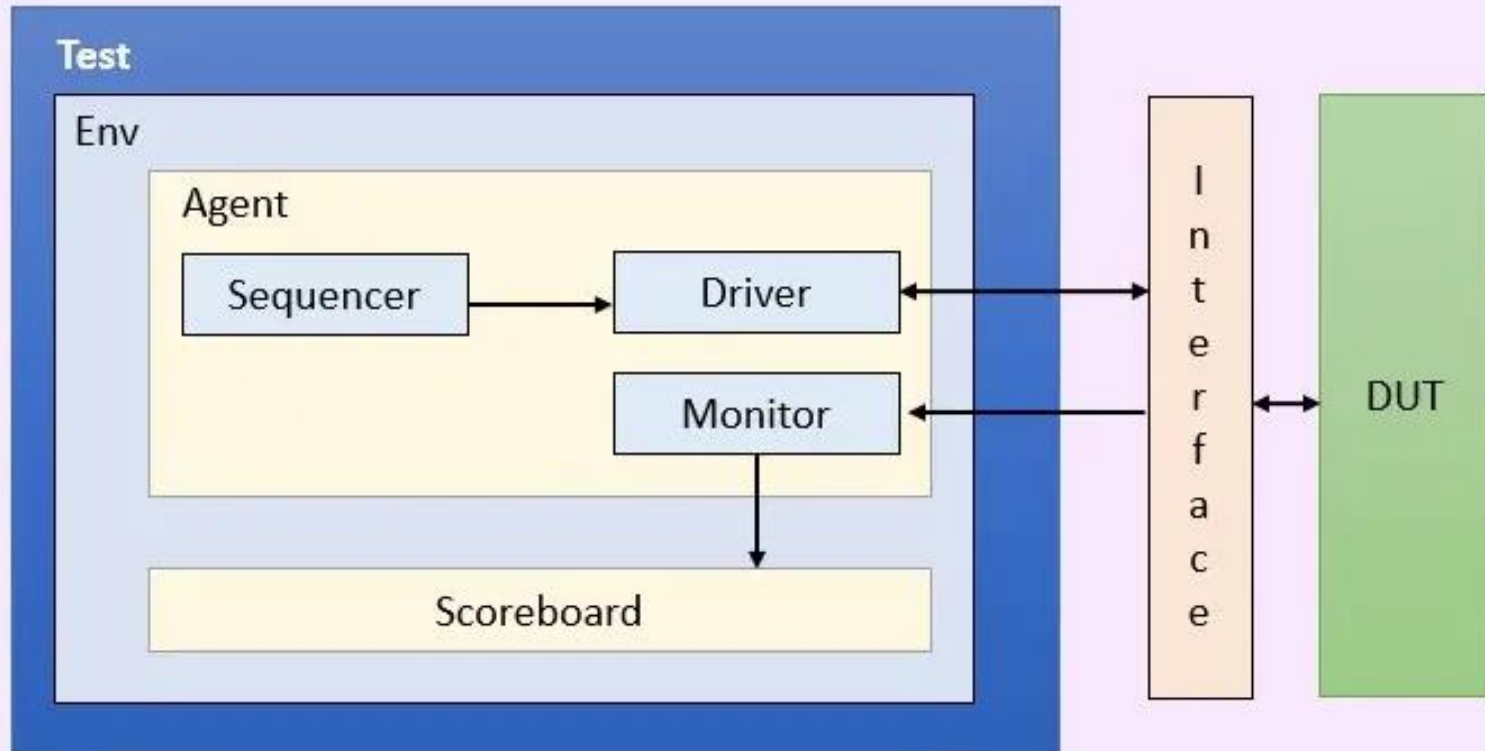


UVM Testbench



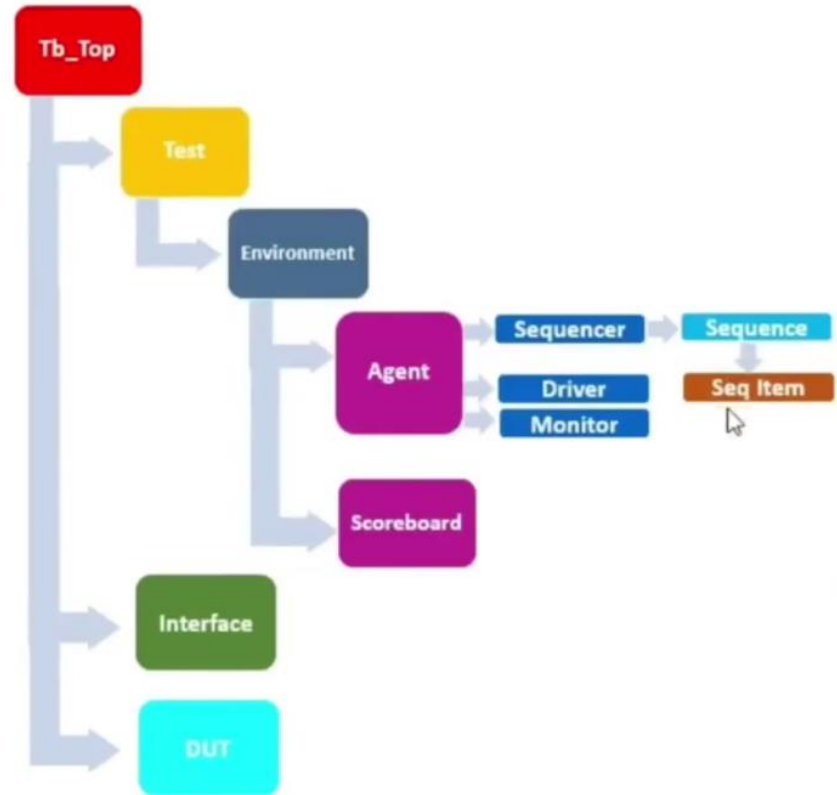
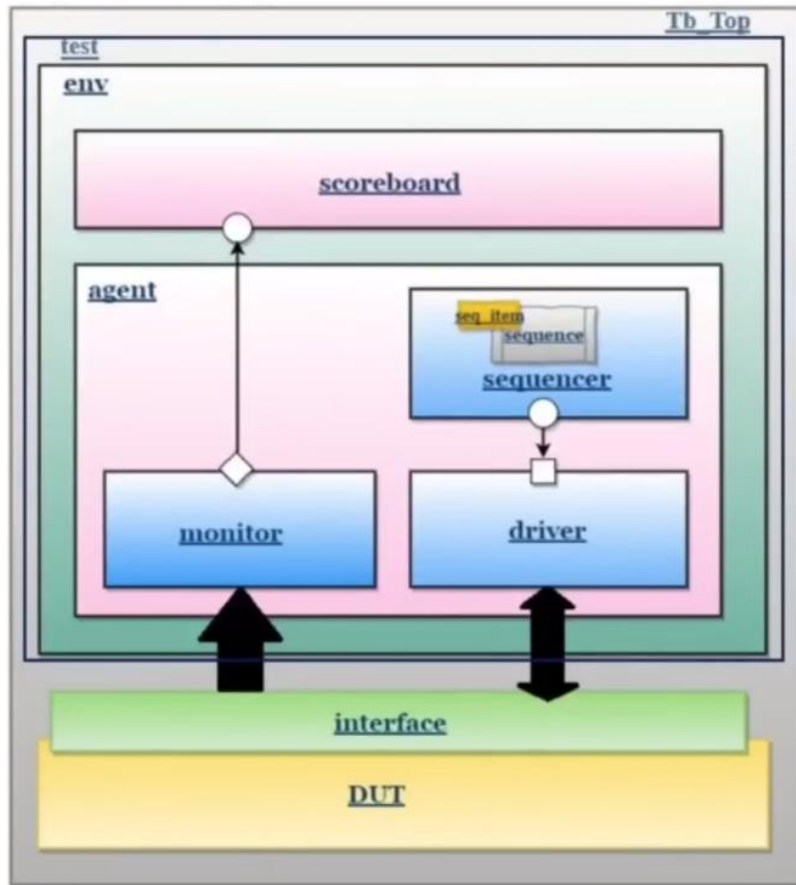
UVM Testbench

TB_top



UVM Testbench

UVM Testbench Architecture



UVM Monitor

```
class wb_bus_monitor extends uvm_monitor;
  `uvm_component_utils(wb_bus_monitor)
  uvm_analysis_port #(wb_txn) wb_mon_ap;
  virtual wb_bus_monitor_bfm m_bfm; //BFM handle
  wb_config m_config;
  // Standard component constructor
  function new(string name, uvm_component parent);
    |
    | super.new(name,parent);
  endfunction
  function void build_phase( uvm_phase phase );
    |
    | wb_mon_ap = new("wb_mon_ap", this);
    | m_config = wb_config::get_config(this); // get config object
    | m_bfm = m_config.WB_mon_bfm; // set local virtual if property
    | m_bfm.proxy = this; //Set BFM proxy handle
  endfunction
  task run_phase(uvm_phase phase);
  m_bfm.run(); //Don't start the BFM until we get to the run_phase
  endtask
  function void notify_transaction(wb_txn item); //Used by BFM to
    | return transactions
    | wb_mon_ap.write(item);
  endfunction : notify_transaction
endclass
```

Source : UVM Cookbook pg. 16

Passive Monitoring

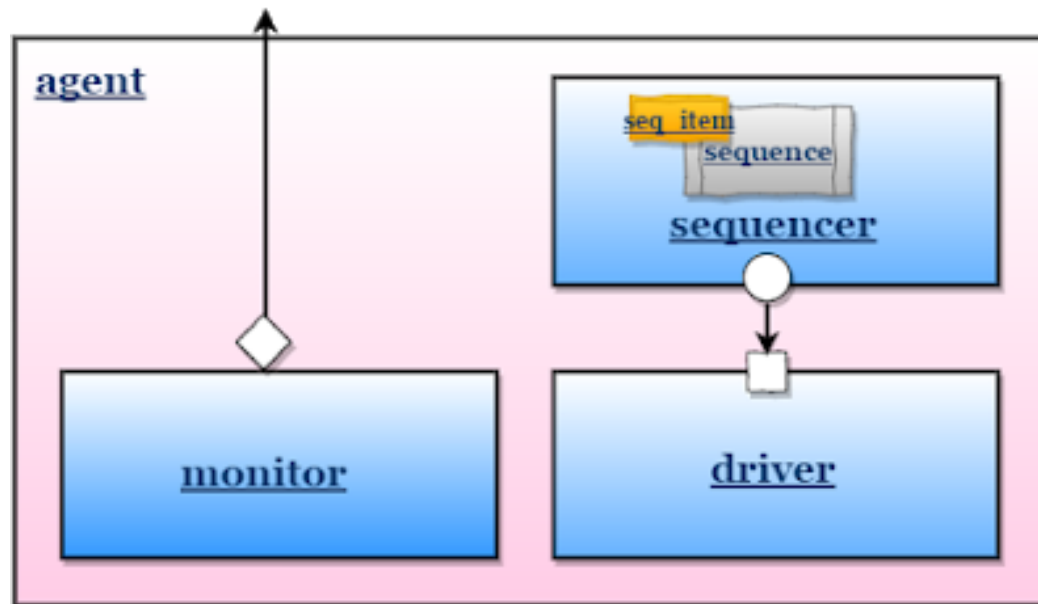
- A UVM monitor should always work in passive mode
- The monitor only observes DUT activity and must not drive or change DUT signals
- Monitor interaction with DUT signals is completely: Read-only
- Observing the DUT should not affect DUT behavior
- The monitor watches the interface signal activity
- UVM phases Passive monitoring ensures safe and non-intrusive verification

Agent

- A user-defined agent is extended from `uvm_agent`,
`uvm_agent` is inherited by `uvm_component`
- An agent typically contains a driver, a sequencer, and a monitor
- Agents can be configured either active or passive

Active Agent

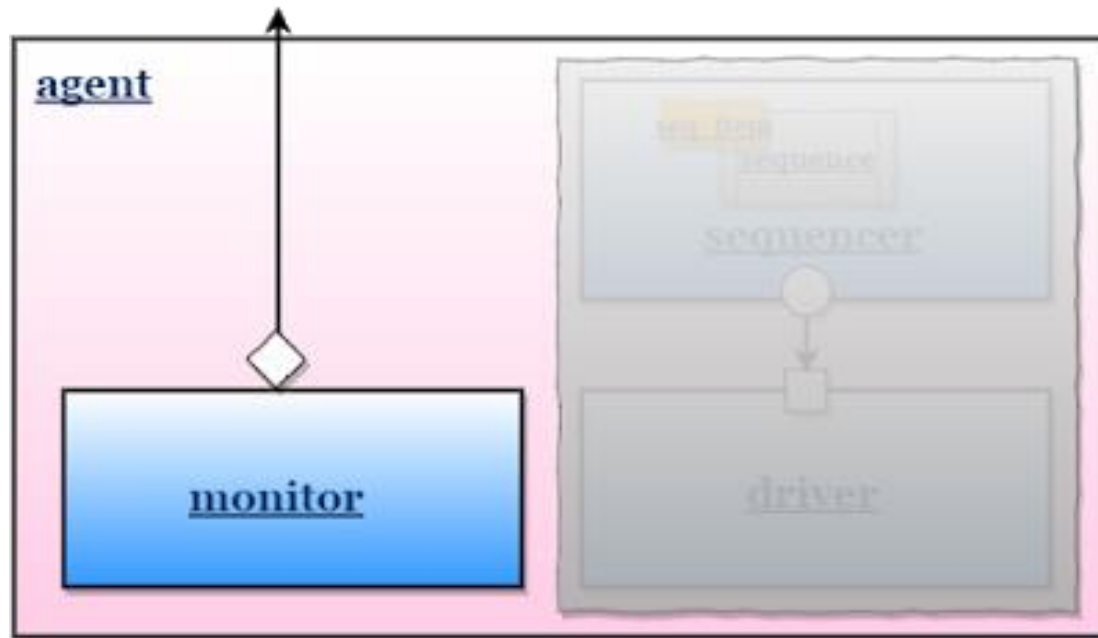
- Active agents generate stimulus and drive to DUT
- An active agent shall consist of all three components: driver, sequencer, and monitor



Source : <https://verificationguide.com/uvm/uvm-agent/>

Passive Agent

- Passive agents sample DUT signals but do not drive them
- A passive agent consists of only the monitor



Source : <https://verificationguide.com/uvm/uvm-agent/>

Scoreboard

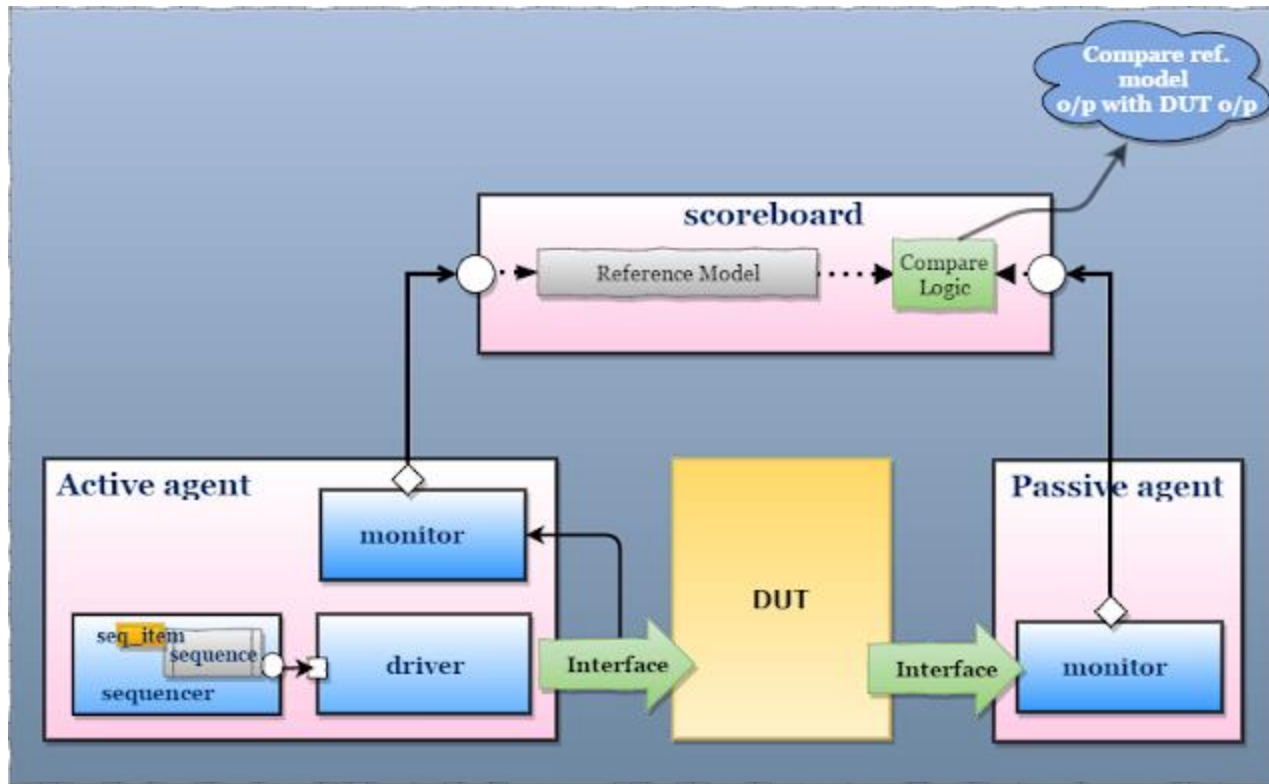
- The scoreboard is written by extending the UVM_SCOREBOARD
- The scoreboard will check the correctness of the DUT by comparing the DUT output with the expected values
- the scoreboard will receive the transactions from the Monitors implemented inside agents
- Monitor and scoreboard will communicate via TLM ports and exports

<https://verificationguide.com/uvm/uvm-scoreboard-example/>

UVM Scoreboard

Scoreboard shall compare the DUT output values with,

- The golden reference values
- The values Generated from the reference model



<https://verificationguide.com/uvm/uvm-scoreboard-example/>

UVM Scoreboard

The scoreboard is written by extending the UVM_SCOREBOARD.

```
✓ class mem_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(mem_scoreboard)

    // new - constructor
    ✓ function new (string name, uvm_component parent);
        super.new(name, parent);
    endfunction : new
endclass : mem_scoreboard
```

<https://verificationguide.com/uvm/uvm-scoreboard-example/>

UVM Environment

- The user-defined environment is derived from `uvm_env`, `uvm_env` is inherited from `uvm_component`
- The environment is the container class
- It contains one or more agents, as well as other components such as the scoreboard, top-level monitor, and checker

```
class mem_model_env extends uvm_env;

    `uvm_component_utils(mem_model_env)

    // new - constructor
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction : new

endclass : mem_model_env
```

<https://verificationguide.com/uvm/uvm-env/>

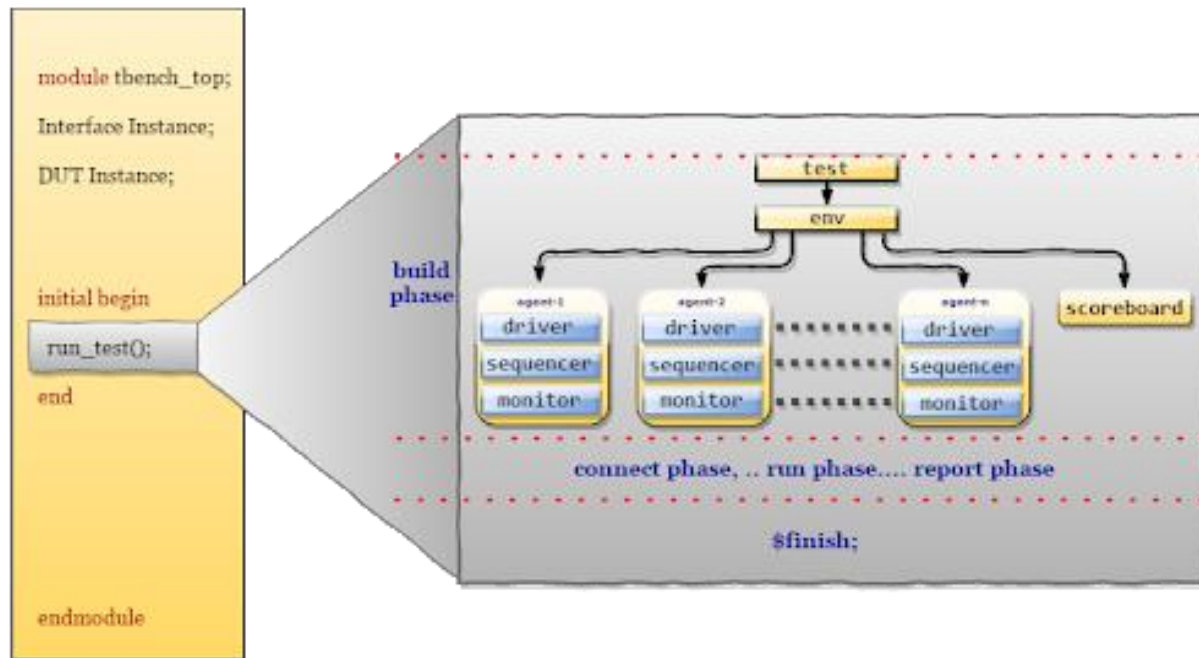
UVM Test

- The user-defined test is derived from `uvm_test`, `uvm_test` is inherited from `uvm_component`
- The test defines the test scenario for the testbench
- test class contains the environment, configuration properties, class overrides etc
- A sequence/sequences are created and started in the test
- The UVM testbench is activated when the `run_test()` method is called, the global `run_test()` task should be specified inside an initial block

```
✓ initial begin  
  |   run_test();  
end
```

UVM TB top

- TestBench top is the module, it connects the DUT and Verification environment components.
- Typical Testbench_top contains, DUT instance, interface instance
- run_test() method, virtual interface set config_db, clock and reset generation logic , wave dump logic



UVM testbench Example

- Refer SystemVerilog testbench for Adder
- Refer SystemVerilog testbench for Memory Model
- UVM TestBench to verify Memory Model
 - <https://verificationguide.com/uvm/uvm-testbench-architecture/>
-

<https://verificationguide.com/systemverilog-examples/systemverilog-testbench-examples/>